

Margin-Lens

A Full-Stack Application (Vue 3 + Express.js + MongoDB)

Technical Report — Backend Architecture & System Design

Semester Project Documentation

Project:	MarginLens — Product Margin & Hidden Cost Tracker
Name:	Umeez Anzar (-087) Talha Shafique (-043)
Course:	Web Engineering
Instructor:	Sir Museb
Date:	16th-July-2026

1. Introduction

MarginLens started as a frontend-only prototype for tracking product pricing. That version kept its product list inside the Vue app itself, with no persistence layer and no way to account for the hidden costs that eat into a seller's actual profit — return rates, payment gateway fees, packaging, and shipping.

This iteration turns MarginLens into a full-stack application. A dedicated backend, built with Express.js and MongoDB, now owns all product, category, and report data. The Vue frontend no longer stores or calculates anything on its own — it calls the backend's REST API for every read and write, and the backend performs the margin calculations that are the actual point of the tool.

The core idea behind MarginLens is the distinction between two numbers that look similar but tell very different stories:

- Perceived margin — the margin a seller sees at a glance: $(\text{selling price} - \text{purchase cost}) \div \text{selling price}$.
- Real margin — the same calculation after also subtracting hidden costs (returns, payment fees, packaging, shipping, and miscellaneous costs).

The gap between these two numbers — the margin gap — is what MarginLens is built to surface. This report documents how the system is built, with an emphasis on the backend that performs this calculation, while also covering the frontend that consumes it and the database schema that stores it.

2. Technology Stack

2.1 Backend

Technology	Role
Node.js	JavaScript runtime the backend runs on
Express.js	Handles HTTP routing and responses for the REST API
MongoDB	Document database storing products, categories, and reports
Mongoose	Defines schemas and mediates all reads/writes to MongoDB
cors	Allows the frontend dev server (different port) to call the API
dotenv	Loads the MongoDB connection string from a local .env file
Exchange Rate API	External API used to convert USD prices into PKR

2.2 Frontend

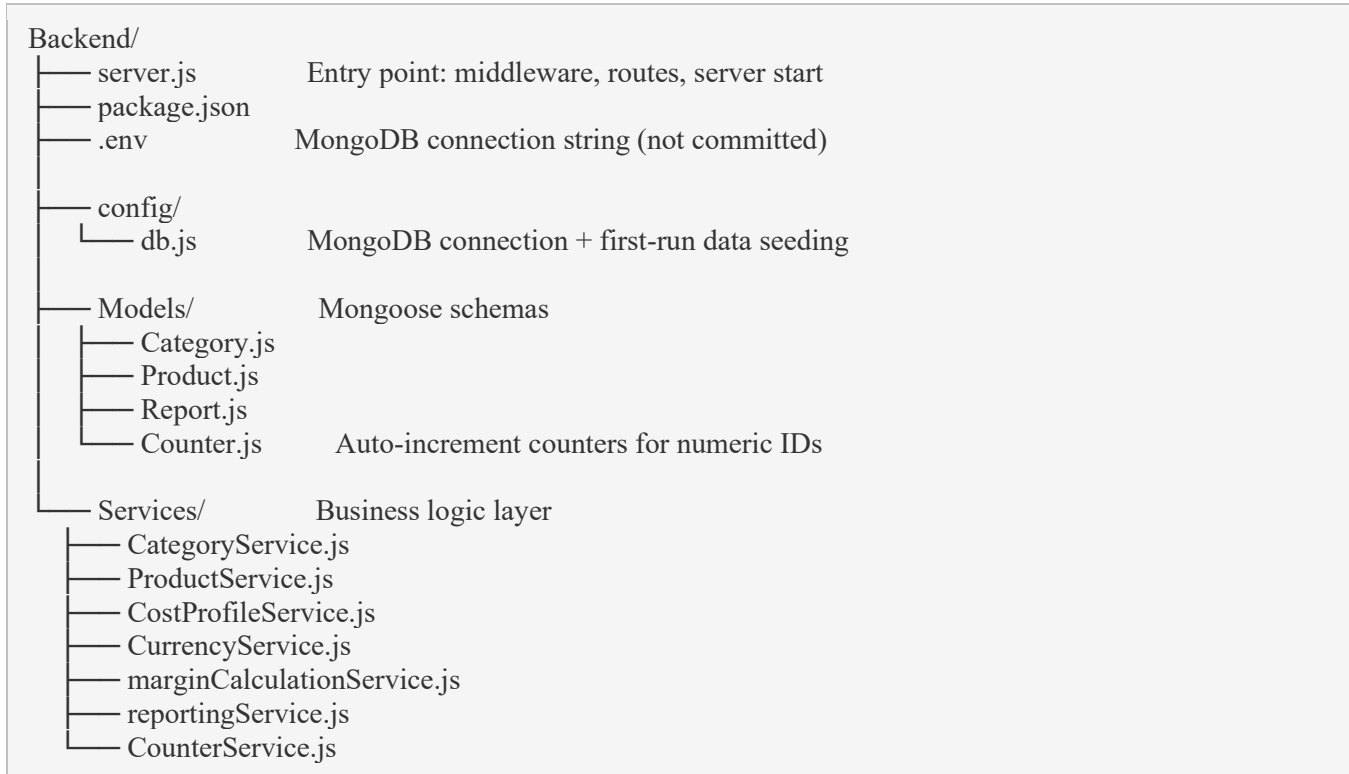
Technology	Role
Vue 3	Component-based frontend framework
Vite	Development server and build tool
Vue Router	Client-side SPA routing

Technology	Role
Axios	Sends HTTP requests to the backend REST API

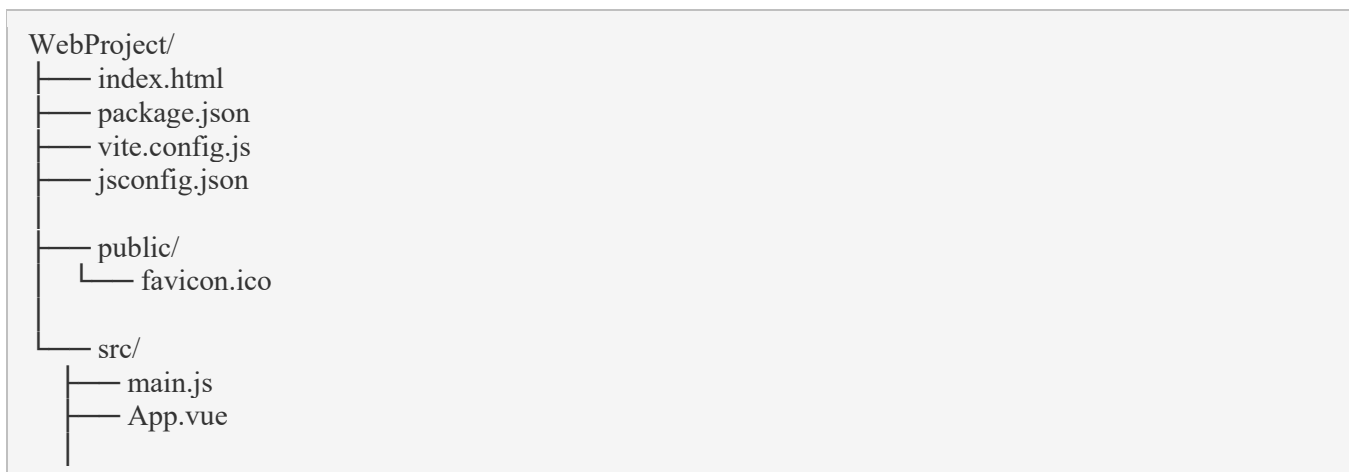
3. Project Structure

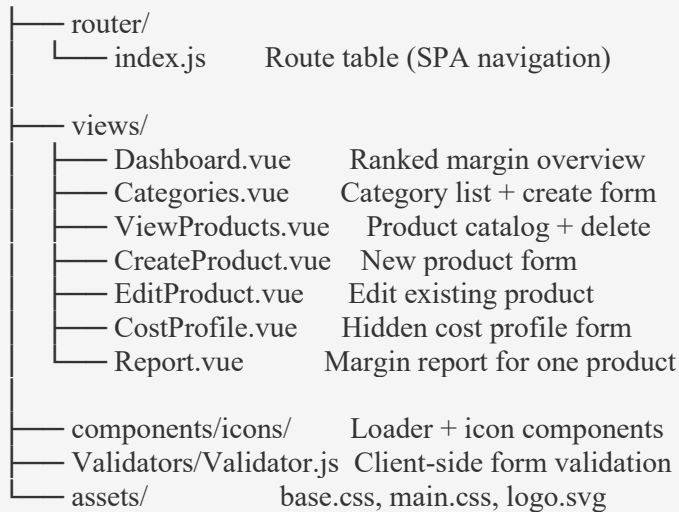
The project is organized as two independent folders in a single repository, each with its own package.json and dependencies.

3.1 Backend structure



3.2 Frontend structure





4. Backend Architecture (Express.js)

Unlike a typical routes/controllers/models split, this backend keeps things flatter: `server.js` defines the routes directly and hands off immediately to a Services layer, which is the only layer allowed to talk to the Mongoose Models. This keeps the HTTP concerns (`server.js`) fully separate from the business logic (Services), while avoiding an extra controller layer the project doesn't need.

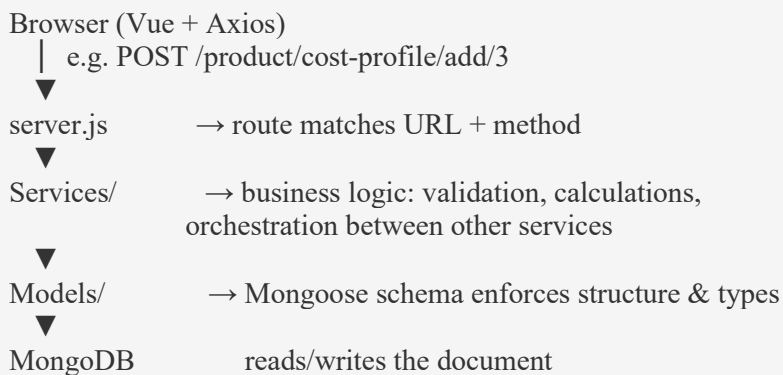
4.1 Middleware

Two middlewares run on every request in `server.js`:

- `cors()` — allows the Vite dev server (port 5173) to call the API on port 3000, which browsers block by default across origins.
- `express.json()` — parses JSON request bodies so `req.body` is populated on POST and PUT requests.

4.2 Request flow

Every API call follows the same layered path:



4.3 Database connection and auto-seeding

The database connection is opened once in `config/db.js` using a URI read from `process.env.MONGO_URI`, with a local fallback for development. The server only starts listening for requests after the connection succeeds, and `connectDB()` also seeds a handful of starter categories and one sample product the first time it runs against an empty database — subsequent restarts detect existing documents and skip seeding.

```
async function connectDB() {
  const uri = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/marginlens';
  await mongoose.connect(uri);
  console.log('MongoDB connected');
  await seedInitialData();
}

// Only seeds if the collections are empty, so it's safe on every restart.
async function seedInitialData() {
  const categoryCount = await Category.countDocuments();
  if (categoryCount === 0) {
    await Category.insertMany([ ...starter categories... ]);
  }
  const productCount = await Product.countDocuments();
  if (productCount === 0) {
    await Product.create({ ...sample product... });
  }
}
```

5. Database Design and Schema

The database is accessed through Mongoose, which enforces a schema on top of MongoDB's normally schema-less collections — every document is validated against a defined structure before it is saved. There are four collections: categories, products, reports, and counters.

5.1 categories collection

Field	Type	Explanation
id	Number	Application-level unique ID, generated by the Counter service
name	String	Category name; checked for duplicates (case-insensitive) before insert

5.2 products collection

The core entity. Each product embeds its own hidden cost profile and latest report directly on the document, so the most recent figures can be read without a second query.

Field	Type	Explanation
id	Number	Unique application-level ID (from Counter)

Field	Type	Explanation
categoryId	Number	Reference to a category's id
name	String	Product name
sellingPrice	Number	Selling price in the product's currency
purchaseCost	Number	Purchase/acquisition cost in the product's currency
currencyCode	String	"PKR" or "USD"
createdAt	String	ISO timestamp set on creation
hiddenCostProfile	Subdocument	Embedded cost profile (return rate, fees, packaging, shipping, misc); null until set
latestReport	Subdocument	Embedded copy of the most recently generated report; null until generated

5.3 reports collection

Every time a report is generated it is appended here, giving a full history per product even though the product document only keeps the latest one.

Field	Type	Explanation
id	Number	Unique report ID (from Counter)
productId	Number	The product this report was calculated for
productName	String	Snapshot of the product name at calculation time
perceivedMargin	Number	Margin before hidden costs, as a percentage
realMargin	Number	Margin after hidden costs, as a percentage
marginGap	Number	$\text{perceivedMargin} - \text{realMargin}$
totalHiddenCost	Number	Sum of all hidden costs in the profile
sellingPricePkr / purchaseCostPkr	Number	Prices used for the calculation, converted to PKR if needed
convertedCostUsed / exchangeRateUsed	Number	Populated only when the product's currency was USD
calculatedAt	String	ISO timestamp of the calculation

5.4 counters collection

A single-purpose collection used to hand out sequential numeric IDs (categoryId, productId, reportId), since the project deliberately avoids exposing MongoDB's own ObjectId values to the frontend.

Field	Type	Explanation
_id	String	The counter's name, e.g. "productId"
seq	Number	The last value issued; incremented atomically on each use

5.5 Key design decisions

- Numeric IDs over ObjectIds: a Counter collection issues short sequential numbers so frontend routes (e.g. /product/edit/3) stay simple, at the cost of one extra atomic update per creation.
- Embedding over referencing for "latest" data: hiddenCostProfile and latestReport are embedded directly in the product document so the dashboard and product views can read current state in a single query, while the full calculation history still lives separately in the reports collection.
- Uniqueness enforced at the schema level: id fields are marked unique so MongoDB itself rejects duplicates rather than relying solely on application code.
- Currency stored, not normalized at write time: prices are kept in their original currency (PKR or USD) and only converted at report-generation time, so the source values are never silently altered.

6. Business Logic Layer

The Services folder is where MarginLens' actual purpose lives. Three services work together to turn a product and its cost profile into a margin report.

6.1 Margin calculation service

A small, dependency-free module of pure functions — no database or network calls — which makes the core margin math easy to reason about and test in isolation.

```
function calculatePerceivedMargin(sellingPrice, purchaseCost) {
  return ((sellingPrice - purchaseCost) / sellingPrice * 100).toFixed(2);
}

function calculateTotalHiddenCost(sellingPrice, profile) {
  const returnCost = sellingPrice * (profile.returnRatePercent / 100);
  const paymentFee = sellingPrice * (profile.paymentFeePercent / 100);
  return returnCost + paymentFee + profile.packagingCost
    + profile.shippingCost + profile.miscCost;
}

function calculateRealMargin(sellingPrice, purchaseCost, totalHiddenCost) {
  return ((sellingPrice - purchaseCost - totalHiddenCost)
    / sellingPrice * 100).toFixed(2);
}
```

```
function calculateMarginGap(perceivedMargin, realMargin) {
  return (perceivedMargin - realMargin).toFixed(2);
}
```

6.2 Currency conversion service

For products priced in USD, CurrencyService calls an external exchange rate API to fetch the current USD → PKR rate, then converts both the selling price and purchase cost before any margin calculation happens. Network or API failures are caught and re-thrown as a clear error rather than allowed to produce a silently wrong report.

6.3 Reporting service (orchestration)

reportingService.generateAndSaveReport() ties everything together: it loads the product, converts currency if needed, calls the margin calculation functions in sequence, assigns the report a new ID from the counter service, saves it to the reports collection, and finally caches the result on the product document as latestReport.

1. Load the product and its hiddenCostProfile
2. If currencyCode is USD → convert sellingPrice and purchaseCost to PKR
3. perceivedMargin = calculatePerceivedMargin(sellingPrice, purchaseCost)
4. totalHiddenCost = calculateTotalHiddenCost(sellingPrice, profile)
5. realMargin = calculateRealMargin(sellingPrice, purchaseCost, totalHiddenCost)
6. marginGap = calculateMarginGap(perceivedMargin, realMargin)
7. Save the report to the reports collection
8. Cache the report on the product as latestReport

getRankedProducts() reuses this cached data to build the dashboard view: it filters out products with no report yet, then sorts the rest by marginGap descending, so the products where hidden costs distort profitability the most appear at the top.

7. REST API Endpoints

Method	Endpoint	Description
GET	/categories	List all categories
GET	/products	List all products, each with its category attached
GET	/product/cost-profile/get/:id	Fetch a product's hidden cost profile
GET	/product/report/:id	Fetch the latest margin report for a product
GET	/products/ranked	List products with a report, ranked by margin gap

Method	Endpoint	Description
POST	/categories/add	Create a category
POST	/products/add	Create a product
POST	/product/cost-profile/add/:id	Save a cost profile and generate a new report
PUT	/product/update/:id	Update an existing product
DELETE	/product/delete/:id	Delete a product

Responses use standard HTTP status codes: 200 for successful reads, 201 for successful creation, 204 for successful deletion, and 404 when a requested product or report doesn't exist.

8. Frontend Overview

While the backend is the focus of this report, the Vue frontend is what makes the margin data usable day to day.

8.1 Views and routing

Route	View	Purpose
/	Dashboard.vue	Shows products ranked by margin gap
/products	ViewProducts.vue	Full product catalog with delete
/product/create	CreateProduct.vue	Create a new product
/product/edit/:id	EditProduct.vue	Edit an existing product
/categories	Categories.vue	List and create categories
/cost-profile/:id	CostProfile.vue	Enter a product's hidden cost profile
/product/report/:id	Report.vue	View a product's margin report

8.2 Client-side validation

Validator.js provides simple rule checks — required fields, positive numeric values, non-empty names — used by the product, category, and cost profile forms before a request is sent, catching obviously invalid input without a round trip to the API.

8.3 API communication

Every view calls the backend directly with Axios rather than going through a shared API module, using a hard-coded `http://localhost:3000` base URL. Centralizing these calls into a single `api.js` service (as MarginLens' own frontend README template suggests) would be a natural next refactor.

9. End-to-End Example: Generating a Margin Report

Tracing one user action through every layer shows how the pieces fit together:

- The user fills in return rate, payment fee, packaging, shipping, and misc. cost on the CostProfile.vue form and submits.
- Axios sends POST /product/cost-profile/add/:id with the cost profile in the request body.
- server.js matches the route and calls CostProfileService.upsertCostProfile(), which validates and saves the profile onto the product via ProductService.
- server.js then calls reportingService.generateAndSaveReport() for the same product.
- If the product's currency is USD, CurrencyService fetches the live exchange rate and converts both prices to PKR.
- marginCalculationService computes perceivedMargin, totalHiddenCost, realMargin, and marginGap.
- The new report is saved to the reports collection and cached as latestReport on the product.
- The frontend navigates to Report.vue, which fetches GET /product/report/:id and displays the calculated figures.

10. Conclusion

This deliverable turned MarginLens from a frontend prototype into a working full-stack system. The Express backend now owns all persistence and, more importantly, owns the margin calculation logic that is the actual value of the tool — separating the simple, easily-miscalculated "perceived" margin from the real margin after hidden costs. MongoDB, accessed through Mongoose, gives the project a schema-enforced but flexible store for products, categories, and report history, while the Vue frontend focuses purely on collecting input and displaying results. The clearest next steps are centralizing the frontend's scattered Axios calls into one API module and moving the hard-coded backend URL into an environment variable.